Clock Divider

srcClockDiveder.sv | tbClockDiveder.sv | Untitled 4* | button_debouncer.v

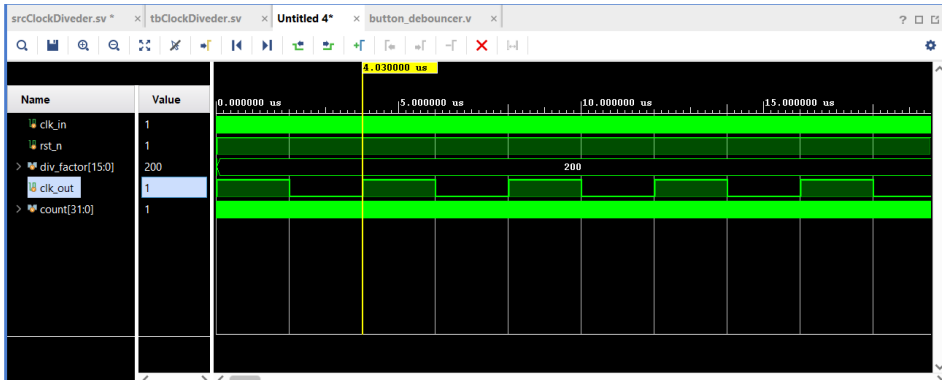
C:/Users/seyma/OneDrive/Yonga/HW1_adder_substractor/srcClockDiveder.sv

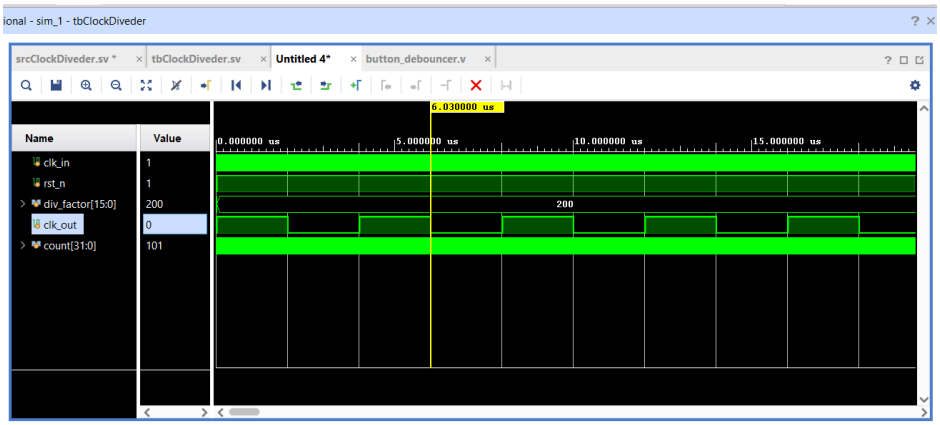
```
module srcClockDiveder (

    input logic clk_in,
    input logic rst_n,
    input logic [15:0] div_factor,
    output logic clk_out

);
int count=0;
logic [15:0] div_factor_temp;
always_comb begin
    div_factor_temp= (div_factor<2) ? 2 : div_factor;       //avoiding 0 and 1 values for div_factor

end

always_ff @(posedge clk_in ,negedge rst_n) begin        //asynchronous reset
    if(rst_n==0) begin
        clk_out<=0;
        end
    else begin
        count <= (count<(div_factor_temp-1)) ? (count+1) : 0;       // to prevent infinite loop inequality condition statement is utilized.
        clk_out<= (count<(div_factor_temp)/2) ? 1: 0;
        end

end

endmodule
```

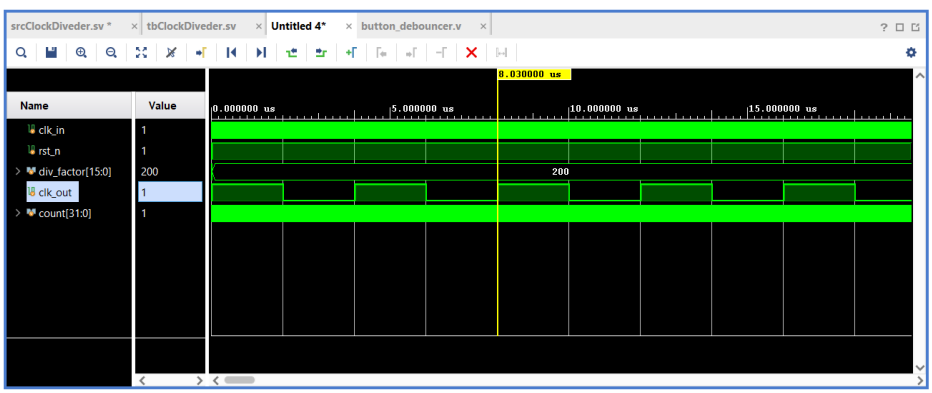
srcClockDiveder.sv | tbClockDiveder.sv | Untitled 4* | button_debouncer.v

C:/Users/seyma/OneDrive/Yonga/HW1_adder_substractor/tbClockDiveder.sv

```
    .clk_out(clk_out)

);

always #10 clk_in=~clk_in;
initial begin
    clk_in=0;
    rst_n=0;
    #20;
    rst_n=1;
    //First give a known value
    div_factor=200;
    #100000;
    //give random values to test different scenarios
    div_factor=$urandom();
    #100000;
    div_factor=$urandom();
    #100000;
    div_factor=$urandom();
    #100000;
    div_factor=$urandom();
    #100000;
    div_factor=$urandom();
    #100000;
    //give 0 and 1 to test behaviour of combinational circuit.
    div_factor=0;
    #100000;
    div_factor=1;
    #100000;
    div_factor=200;

end
```







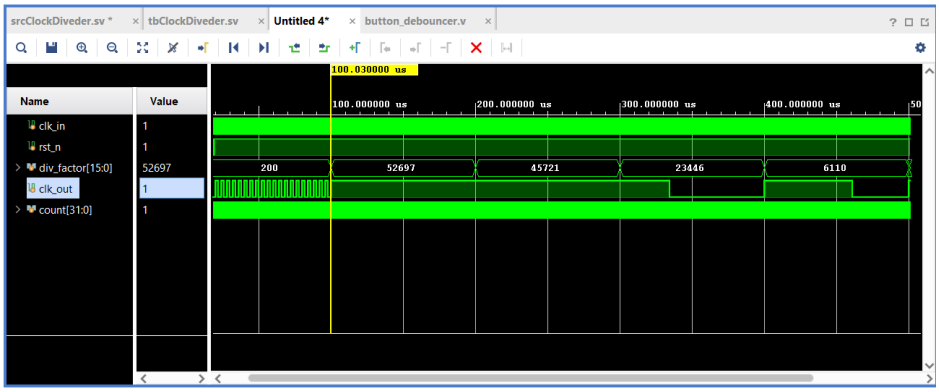
clk_in toggles every 10 nanoseconds. This means the clock period is 20 nanoseconds, and a posedge occurs every 20 nanoseconds.

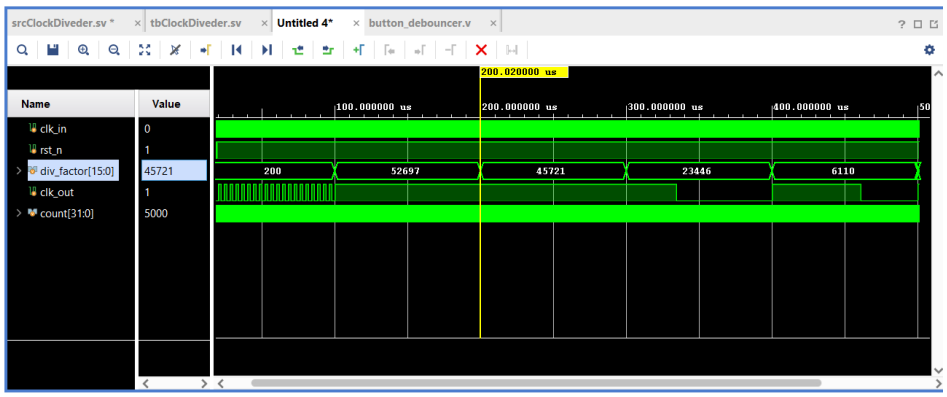
In this scenario, div_factor is 200. clk_out will toggle after 100 posedge iterations (div_factor_temp / 2).

Therefore, after 100 iterations × 20 nanoseconds = 2000 nanoseconds, clk_out will toggle.

The signal will remain high for 2000 nanoseconds and low for 2000 nanoseconds, which corresponds to exactly a 50% duty cycle.

The counter will reset to zero every 4000 nanoseconds.





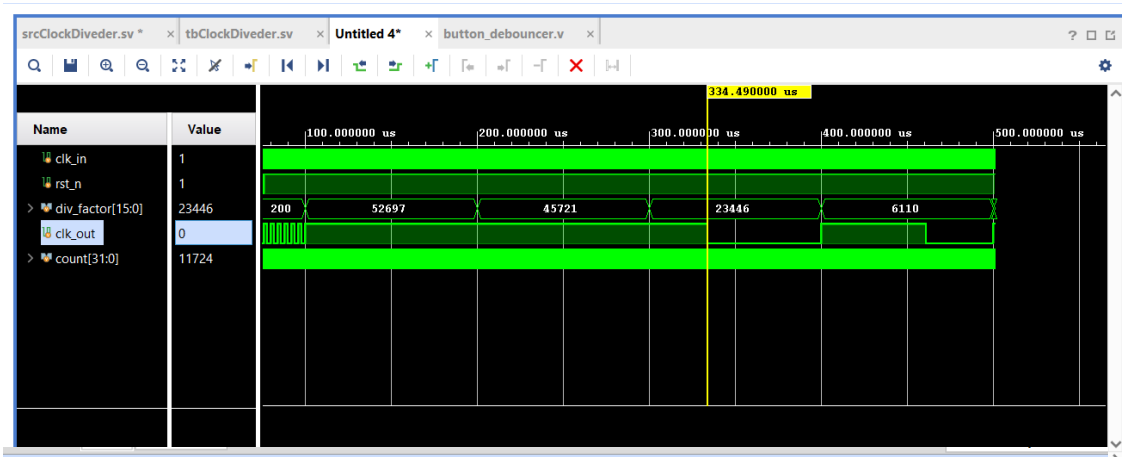
When div_factor is 52697, clk_out will toggle after 26,348 iterations (div_factor / 2).

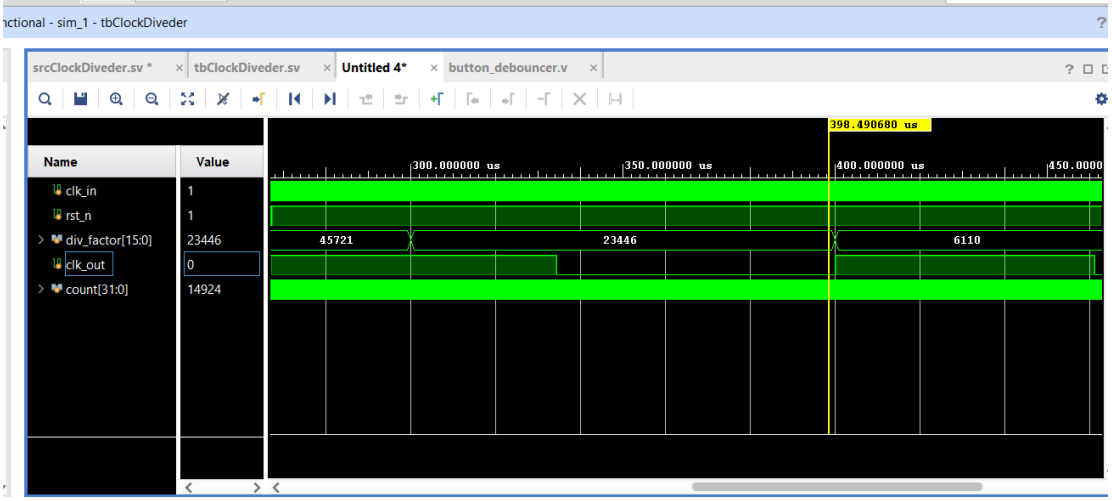
Since each iteration corresponds to one posedge of clk_in, and each posedge occurs every 20 nanoseconds, the toggle time is:

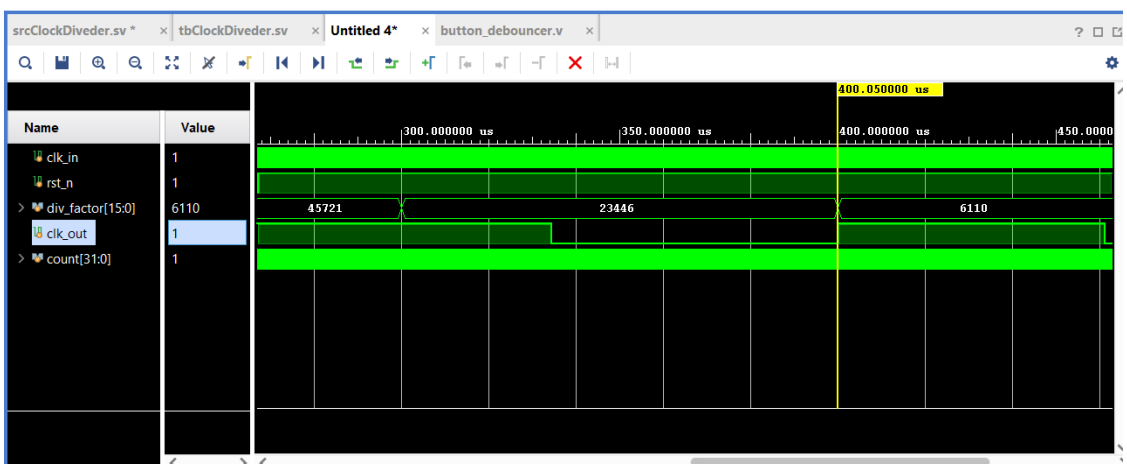
26,348 × 20 nanoseconds = 526,960 nanoseconds.

Therefore, clk_out will toggle after 526,960 nanoseconds.

However, the time delay for the div_factor change is 200,000 nanoseconds. Because 200000 ns is smaller than 526960 ns, clk_out does not toggle before div_factor changes again.



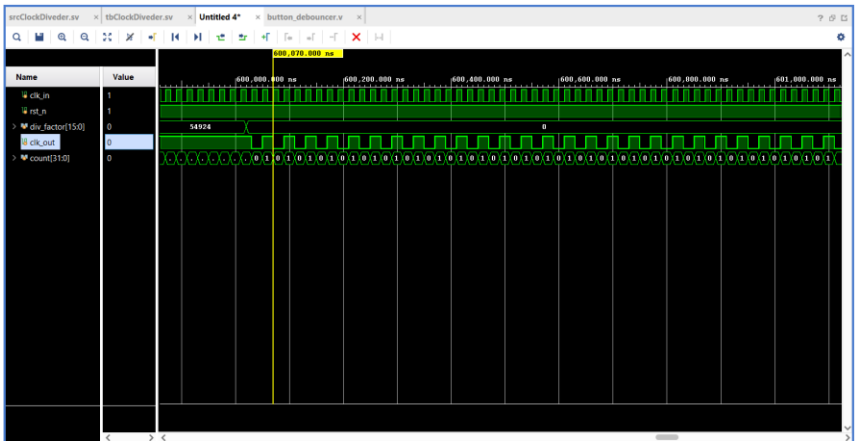


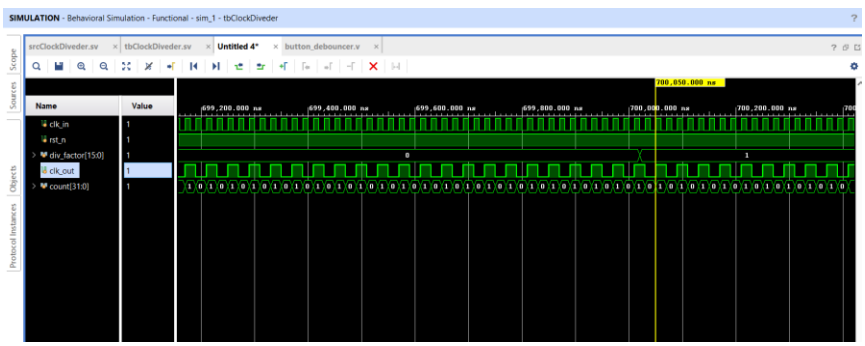


div_factor changes immediately and becomes smaller than the current counter value. For example, div_factor = 6110 while count is already greater than 14924. With this line:

count <= (count < (div_factor_temp-1)) ? (count+1) : 0;

the condition becomes false, so count is set to 0 and then starts counting up again from 0.





Zero and one scenarios are given above. It is shown that when div_factor becomes 0 or 1, the period of clk_out is fixed to two times the period of clk_in.